

2. 步骤一：资源提交与接入校验

文档入库可以从 HTTP、SDK 或本地嵌入式 Client 发起。HTTP 模式下，前端或业务系统通常先调用 `temp_upload` 上传文件，再把 `temp_file_id` 交给 `/api/v1/resources`；远程 URL、代码仓库 URL、飞书链接等也可以直接作为 `path` 提交。

入口	关键参数	说明
POST <code>/api/v1/resources/temp_upload</code>	<code>file</code>	保存上传文件，返回 <code>temp_file_id</code> 和 <code>original_filename</code> 。
POST <code>/api/v1/resources</code>	<code>path</code> 或 <code>temp_file_id</code>	提交资源处理任务。二者必须至少提供一个。
SDK <code>add_resource</code>	<code>path</code> 、 <code>to</code> 、 <code>parent</code> 、 <code>wait</code> 、 <code>timeout</code> 等	本地或 HTTP Client 封装入库调用。

参数	作用	处理逻辑
<code>path</code>	远程资源地址或本地路径	HTTP 直连模式要求远程来源；SDK/本地模式可允许本地路径。
<code>temp_file_id</code>	临时上传文件编号	由 <code>resolve_uploaded_temp_file_id</code> 解析为上传目录下的真实文件。
<code>to</code>	精确目标 URI	例如 <code>viking://resources/product/manual</code> ；不能与 <code>parent</code> 同时使用。
<code>parent</code>	目标父目录	<code>TreeBuilder</code> 在该父目录下根据文档名生成最终 URI。
<code>folder_path</code>	知识管理 UI 的虚拟文件夹	不直接改变内部资源树，可用于知识文档记录归类。
<code>instruction/reason</code>	处理提示	会作为语义摘要阶段的提示信息，提高 L0/L1 与场景相关性。
<code>wait/timeout</code>	是否等待异步处理完成	<code>wait=true</code> 时等待队列完成；超时抛 <code>DeadlineExceededError</code> 。
<code>include/exclude/ignore_dirs/strict</code>	目录或仓库解析策略	透传给 <code>DirectoryParser</code> 和扫描逻辑。
<code>watch_interval</code>	资源监控周期	大于 0 时创建或更新 <code>watch task</code> ，用于后续自动同步。

安全边界

HTTP Server 不接受任意主机本地路径作为 `path`，除非文件先经过 `temp_upload`。这个设计避免外部调用方通过入库接口读取服务端文件系统。

3. 步骤二：ResourceService 编排入库请求

`ResourceService.add_resource` 是资源入库的业务编排层。它不直接解析文件，而是负责参数治理、目标 URI 约束、知识文档记录回调、等待队列和 `watch task` 管理。

```
ResourceService.add_resource()
```